Firebase 프로젝트 이전 (Migration) 가이드

새로운 계정(`expand92@gmail.com`)으로 Firebase 프로젝트를 이전하기 위한 전체 절차입니다.

> **기존 프로젝트**: `hunesalesreport` (hclim@hunesion.com)
> **신규 프로젝트**: 예) `hunesalesreport-v2` (expand92@gmail.com)

Phase 1. 사전 준비 — 새 Firebase 프로젝트 생성

1.1 프로젝트 생성

1. 새 계정(`expand92@gmail.com`)으로 Firebase 콘솔에 로그인
2. `프로젝트 추가` 클릭 → 새 프로젝트 생성 (예: `hunesalesreport-v2`)
3. **Blaze(종량제) 요금제 업그레이드** — Phase 3 방법 A(GCP 내보내기/가져오기) 사용 시 필수

1.2 서비스 활성화

서비스	활성화 방법	주의사항
Firestore Database	좌측 메뉴 > `데이터베이스 만들기`	리전: `asia-northeast3` (서울) — 기존과 동일하게
Authentication	`시작하기` > `Sign-in method` > **이메일/비밀번호** 활성화	
Hosting	`시작하기` > 호스팅 초기화	

> **참고**: Firebase Storage는 현재 코드에서 사용하지 않으므로 활성화하지 않아도 됩니다. 향후 파일 업로드 기능이 필요할 때 활성화하세요.

1.3 웹 앱 추가 및 설정 키 발급

1. 설정(톱니바퀴) > `프로젝트 설정` > `일반` 탭 하단 > **웹(</>) 앱 추가**
2. 앱 등록 후 나타나는 `firebaseConfig` 정보를 복사해둡니다:

```
apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId
```

Phase 2. Firebase Auth 사용자 이전

기존 프로젝트에 등록된 사용자(이메일/비밀번호) 계정을 새 프로젝트로 이전합니다.

> **중요**: Firebase Auth export/import는 사용자의 **UID를 보존**합니다.
> Firestore `users` 컬렉션의 문서 ID가 UID 기반(`users/{uid}`)이므로, UID가 반드시 유지되어야 Phase 3의 Firestore 데이터와 정합성이 보장됩니다.

2.1 기존 프로젝트에서 사용자 내보내기

```
firebase auth:export users.json --project hunesalesreport --format=json
```

2.2 해시 키 확인

기존 프로젝트의 Firebase 콘솔 > **Authentication** > **Users** 탭 > 우측 상단 메뉴(⋮) > `비밀번호 해시 매개변수` 에서 다음 값을 확인합니다:

- `hash_config.algorithm` (보통 `SCRYPT`)
- `hash_config.base64_signer_key`
- `hash_config.base64_salt_separator`
- `hash_config.rounds`
- `hash_config.mem_cost`

2.3 새 프로젝트로 사용자 가져오기

```
firebase auth:import users.json \
  --project 새_프로젝트ID \
  --hash-algo=scrypt \
  --hash-key=<base64_signer_key> \
  --salt-separator=<base64_salt_separator> \
  --rounds=<rounds> \
  --mem-cost=<mem_cost>
```

2.4 검증

- 새 프로젝트 Firebase 콘솔 > Authentication에서 사용자 목록 확인
- 기존 사용자의 UID가 이전과 동일한지 확인 (Firestore `users` 컬렉션과의 매핑 필수)
- 기존 계정으로 로그인 테스트

> **주의**: `users.json` 파일에는 민감한 사용자 정보(비밀번호 해시 포함)가 포함되어 있으므로, 작업 완료 후 반드시 삭제하세요. `.gitignore` 에 이미 `users.json` 이 등록되어 있습니다.

Phase 3. Firestore 데이터 이전

마이그레이션 대상 컬렉션 전체 목록

`firestore.rules` 및 `src/firebase/services/` 기준, 이전해야 하는 전체 컬렉션 구조입니다:

```
Root Collections (10개):
├── users/{userId}                          ← authService.ts, userService.ts
├── divisions/{divisionId}                  ← divisionService.ts
├── products_master/{productId}             ← productMasterService.ts
├── products/{productId}                    ← 하위 호환용 (productService.ts)
├── reports/{reportId}                      ← reportService.ts
│   ├── /products/{productDocId}            ← productService.ts
│   ├── /division_data/{id}                ← divisionDataService.ts
│   ├── /industry_group_data/{id}          ← industryGroupDataService.ts
│   └── /snapshots/{snapshotId}            ← snapshotService.ts
│       └── /products/{productId}          ← 중첩 서브컬렉션 (3단계)
├── targets/{targetId}                      ← targetService.ts
├── industry_groups/{groupId}               ← industryGroupService.ts
├── product_group_targets/{targetId}        ← productGroupTargetService.ts
├── backlog/{year}                          ← backlogService.ts
│   ├── /products/{docId}
│   ├── /divisions/{docId}
│   └── /industry_groups/{docId}
└── uploadHistory/{historyId}               ← uploadHistoryService.ts
```

방법 A. Google Cloud 내보내기/가져오기 (권장)

타임스탬프, 서브컬렉션, 중첩 구조를 **100% 안전하게** 이전할 수 있습니다.

> **전제조건**: 기존/신규 프로젝트 모두 **Blaze 요금제** 필요

1. 기존 프로젝트 계정으로 Google Cloud Firestore 메뉴 접속
2. `내보내기` 클릭 → 내보낼 Cloud Storage 버킷 선택 후 데이터 백업
3. 해당 Storage 버킷의 IAM 권한에 새 프로젝트의 서비스 계정 주소 추가
   - 역할: `Storage 객체 관리자` (roles/storage.objectAdmin)
   - 서비스 계정 형식: `새_프로젝트ID@appspot.gserviceaccount.com`
4. 새 프로젝트 계정으로 Google Cloud Firestore 접속 > `가져오기` > 이전 버킷 선택 후 복원

방법 B. Node.js 스크립트 사용 ([scripts/migrate-db.js](file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/scripts/migrate-db.js))

로컬에서 두 프로젝트의 서비스 계정 키를 사용하여 데이터를 복사합니다.

스크립트 주요 기능:

- 10개 루트 컬렉션을 순회하며 `copySubcollections()` 로 서브컬렉션을 **재귀적으로 복사** (3단계 중첩까지 자동 처리)
- **배치 처리(Batch Write)**: Firestore 한도인 500건 단위로 자동 분할하여 대량 데이터 처리
- **키 파일 존재 검사**: 실행 전 `old-key.json` , `new-key.json` 존재 여부를 확인하여 누락 시 안내 메시지 출력
- **컬렉션별 에러 격리**: 한 컬렉션에서 오류 발생 시 해당 컬렉션만 실패하고 나머지는 계속 진행

실행 절차:

1. **서비스 계정 키 발급**
   - 기존/신규 프로젝트 각각의 Firebase 콘솔 > 설정 > 서비스 계정 > `새 비공개 키 생성` 클릭
   - 다운로드한 JSON을 프로젝트 루트에 저장:
     - 기존 프로젝트 → `old-key.json`
     - 신규 프로젝트 → `new-key.json`
   - `.gitignore` 에 `*-key.json` 패턴이 이미 등록되어 있어 Git 추적에서 자동 제외됩니다.
2. **패키지 설치 및 실행**

```
# firebase-admin은 현재 package.json에 포함되어 있지 않으므로 별도 설치 필요
npm install firebase-admin
node scripts/migrate-db.js
```

3. **완료 후 키 파일 삭제**

```
rm old-key.json new-key.json
```

> **참고**: 방법 B는 Firestore Timestamp 객체를 자동으로 보존합니다. 대량 데이터 시 배치 처리로 Firestore 쓰기 할당량 초과를 방지하지만, 데이터가 수만 건 이상이라면 방법 A를 권장합니다.

Phase 4. 소스 코드 변경

4.1 환경변수 파일 수정

[.env](file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/.env)와 [.env.production](file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/.env.production) 두 파일 모두 수정합니다:

```
VITE_FIREBASE_API_KEY=새_프로젝트의_API_키
VITE_FIREBASE_AUTH_DOMAIN=새_프로젝트ID.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=새_프로젝트ID
VITE_FIREBASE_STORAGE_BUCKET=새_프로젝트ID.firebasestorage.app
VITE_FIREBASE_MESSAGING_SENDER_ID=새_프로젝트의_SENDER_ID
VITE_FIREBASE_APP_ID=새_프로젝트의_APP_ID

# 관리자 이메일 (아래 주의사항 필독)
VITE_ADMIN_EMAIL=관리자_이메일_주소
```

> **`VITE_ADMIN_EMAIL` 동작 방식 (중요)**
>
> 이 값은 `src/firebase/services/authService.ts` 에서 다음과 같이 사용됩니다:
>
> - **회원가입 시**: 이 이메일로 가입하면 자동으로 `role: 'admin'` , `status: 'approved'` 부여
> - **로그인 시**: 이 이메일 계정이 admin이 아닌 상태라면 자동으로 권한 상향 조정
>
> 따라서:
>
> - 기존 관리자(`hclim@hunesion.com`)를 유지하려면 값을 변경하지 마세요.
> - 새 관리자로 변경하려면 해당 이메일이 Firebase Auth에 등록되어 있어야 합니다.

4.2 Firebase 프로젝트 설정 파일 수정

파일	변경 내용
[.firebaserc] (file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/.firebaserc)	`"default": "hunesalesreport"` → `"default": "새_프로젝트ID"`
[firebase.json] (file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/firebase.json) (line 23)	`"site": "hunesalesreport"` → `"site": "새_프로젝트ID"`

> **`firebase.json` 의 `functions` 섹션 정리**
>
> 현재 `firebase.json` 에 `functions` 설정이 있지만, `functions/` 디렉토리는 존재하지 않고 소스 코드에서도 Cloud Functions를 호출하는 코드가 없습니다.
> `npm run deploy:all` 실행 시 functions 배포 오류가 발생할 수 있으므로, **해당 섹션을 제거하는 것을 권장합니다:**
>
> ```
> // firebase.json에서 아래 블록 제거
> - "functions": [
> -   {
> -     "source": "functions",
> -     "codebase": "default",
> -     "ignore": [...],
> -     "predeploy": [...]
> -   }
> - ],
> ```

4.3 Firebase CLI 프로젝트 전환

```
firebase logout
firebase login                  # expand92@gmail.com 계정으로 로그인
firebase use --add              # 새 프로젝트 ID를 선택하고 alias: default 지정
```

Phase 5. Security Rules, Indexes 및 웹 호스팅 배포

소스에 포함된 Firestore 규칙, 인덱스, 프론트엔드 앱을 새 프로젝트에 배포합니다.

```
# 새로운 프로젝트가 선택된 상태인지 확인
firebase use

# Security Rules + Indexes + React 앱 빌드 및 전체 배포
npm run deploy:all
```

배포 대상 파일:

- [firestore.rules](file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/firestore.rules) — 10개 컬렉션의 보안 규칙 (382줄)
- [firestore.indexes.json](file:///Volumes/hclim_SSD/sales_data_weekly/sales-data-weekly/firestore.indexes.json) — 복합 인덱스 3개 (`targets` , `product_group_targets`)
- `dist/` — React 앱 빌드 결과물 (Hosting)

> **참고**: `firebase.json` 에서 `functions` 섹션을 제거하지 않았다면, `deploy:all` 대신 개별 배포를 사용하세요:
>
> ```
> npm run build
> firebase deploy --only firestore:rules,firestore:indexes,hosting
> ```

Phase 6. 마이그레이션 후 검증

체크리스트

#	검증 항목	방법	통과 기준
1	**로그인**	기존 계정(이메일/비밀번호)으로 로그인	정상 로그인 성공
2	**관리자 권한**	`VITE_ADMIN_EMAIL` 계정으로 로그인 후 관리자 메뉴 접근	admin 페이지 진입 가능
3	**대시보드 수치**	메인 대시보드 KPI 수치 비교	기존 프로젝트와 동일
4	**컬렉션 문서 수**	Firestore 콘솔에서 각 루트 컬렉션 문서 수 비교	기존과 일치

#	검증 항목	방법	통과 기준
5	**서브컬렉션**	`reports/{year}/products` 등 하위 데이터 확인	데이터 존재 확인
6	**중첩 서브컬렉션**	`reports/{year}/snapshots/{id}/products` 확인	3단계 데이터 존재
7	**Security Rules**	로그아웃 상태에서 Firestore 직접 접근 시도	접근 차단 확인
8	**엑셀 업로드**	새 매출 데이터 엑셀 업로드	정상 파싱 및 저장
9	**부문별 리포트**	부문별 리포트 페이지 진입	차트/테이블 정상 표시
10	**제품별 리포트**	제품별 리포트 페이지 진입	차트/테이블 정상 표시
11	**달성률 분석**	달성률 분석 페이지 진입	목표 대비 실적 정상 표시
12	**관리자 기능**	사용자/부문/제품/목표 관리 페이지	CRUD 동작 정상
13	**Hosting URL**	`https://새_프로젝트ID.web.app` 접속	페이지 정상 로드

Phase 7. 정리 및 마무리

임시 파일 삭제

```
rm -f old-key.json new-key.json users.json
```

기존 프로젝트 처리

- **즉시 삭제하지 말 것** — 최소 1~2주간 신규 프로젝트 안정성 확인 후 결정
- 기존 프로젝트의 Firestore Security Rules를 읽기 전용으로 변경하여 실수로 데이터가 변경되는 것을 방지:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read: if true;
      allow write: if false;
    }
  }
}
```

롤백 계획

마이그레이션 실패 시:

1. `.env` , `.env.production` 파일을 원래 값으로 복원
2. `.firebaserc` 의 `"default"` 를 `"hunesalesreport"` 로 복원
3. `firebase.json` 의 `"site"` 를 `"hunesalesreport"` 로 복원
4. `firebase login` → 기존 계정(`hclim@hunesion.com`)으로 재로그인
5. `npm run deploy:all` 로 기존 프로젝트에 재배포

> **팁**: 롤백을 빠르게 하려면, 변경 전에 설정 파일들을 백업해두세요:
>
> ```
> cp .env .env.backup
> cp .env.production .env.production.backup
> cp .firebaserc .firebaserc.backup
> cp firebase.json firebase.json.backup
> ```